Computational statistics (Group C)

Zhuoyan Jiang, Kalpa Korottage Don, Elsa Mattila

October 2026

GitHub: https://github.com/kalpa2rashmika-sys/Computational-Statistics-Group-C-.git

1 Time Series Bootstrap and the Ladle Estimator

1.1 Problem statement

The objective of this task was to estimate the dimension q of a multivariate time series found in the `timeseries.dat` file provided. This was done by implementing the ladle estimator. In order to calculate the ladle estimator for a dataset one needs to obtain bootstrap samples from the dataset, but this cannot be done with ordinary independent bootstrap methods. Independent bootstrap assumes, as the name refers to, independent observations, but in time series data there is serial dependence between observations. This means that one has to rely on other methods to obtain the bootstrap samples needed. We implemented stationary block bootstrap and fixed block bootstrap for this purpose.

The ladle estimator also requires the joint diagonaliser of the series and the eigenvalue quantity Λ. These come from the second-order source separation (SOS) model, in which the series is whitened and the autocovariance matrices of the whitened series are diagonalised, at a single lag (AMUSE) or jointly over several lags (SOBI). Whitening, the autocovariance matrices, AMUSE and SOBI were implemented to obtain the joint diagonaliser and Λ. The bootstrap samples then show how much the diagonaliser changes from sample to sample, which the ladle estimator uses to find the signal dimension.

The dataset provided for this task consisted of 1000 observations of 10 different signals. No information was given about what the data measured. Most of the series were the same order of magnitude apart from one series, which was two orders of magnitude higher than for the other signals. The ranges of the values varied for the variables with the same order of magnitude, with some variables also having negative values. Figure 1 shows plots of the provided dataset. The plots show that there seems to be serial dependence between observations for all variables. One can also see that it is impossible to distinguish noise from signal by eye, which means that the true signal dimension cannot be estimated by plotting the data alone.

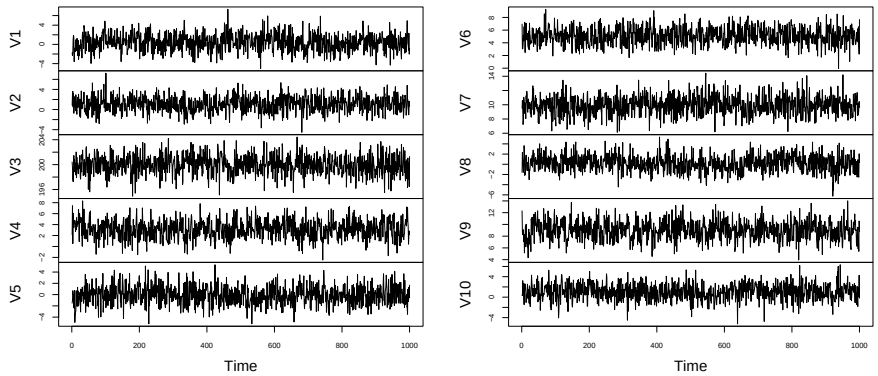


Figure 1: Plot of `timeseries.dat`.

In order to understand, what diagonalisation method and bootstrap method to use, a simulation study was done. The results of this study were then utilised in the application of the ladle estimator on the provided dataset, by determining the settings used for calculation.

1.2 Theory and methodology

1.2.1 Whitening and the SOS model

In the second-order source separation (SOS) model, the observed series is a linear mixture of latent components that are uncorrelated with each other and serially dependent. AMUSE and SOBI whiten the series and then diagonalise the autocovariance matrices of the whitened series. The diagonal elements obtained give the quantity Λ used in φ, which is one of the ladle quantities.

(a) SOS model and whitening

The SOS model assumes that the data is:

$$x_t = \mu + \Omega z_t, \qquad \mathrm{Cov}(z_t) = I_p,$$

where the components of z_t are weakly stationary and uncorrelated with each other at every lag (Nordhausen and Virta, 2018). The covariance matrix of x_t is $\Sigma = \Omega\Omega^\top$, so $\Sigma^{-1/2}\Omega$ is orthogonal and the whitened series is

$$x_t^{st} = \Sigma^{-1/2}(x_t - \mu) = U z_t, \qquad U^\top U = I_p.$$

Whitening removes the scale and the contemporaneous correlation, and it reduces the unknown matrix Ω to the orthogonal matrix U. In the sample, $\bar{x} = \frac{1}{T}\sum_{t=1}^{T} x_t$, $\tilde{x}_t = x_t - \bar{x}$, and

$$\hat{\Sigma} = \frac{1}{T}\sum_{t=1}^{T} \tilde{x}_t \tilde{x}_t^\top = V D V^\top, \qquad \hat{\Sigma}^{-1/2} = V D^{-1/2} V^\top.$$

(b) Autocovariance matrices

The lag-τ autocovariance matrix of the whitened series is

$$\Sigma_\tau = U D_\tau U^\top, \qquad D_\tau = \mathrm{Cov}(z_t, z_{t+\tau}).$$

Because the components are uncorrelated at every lag, D_τ is diagonal, so all Σ_τ share the same diagonalising matrix U. The condition $\mathrm{Cov}(z_t) = I_p$ alone would not give this, since it only describes the components at the same time point. In the sample, the whitened series $\hat{x}_t^{st} = \hat{\Sigma}^{-1/2}\tilde{x}_t$ was used and the matrices were estimated with the $1/T$ normalisation,

$$\hat{\Sigma}_\tau = \frac{1}{T}\sum_{t=1}^{T-\tau} \hat{x}_t^{st} \hat{x}_{t+\tau}^{st\top}.$$

Since Σ_τ is symmetric under the model, the antisymmetric part of $\hat{\Sigma}_\tau$ is only sampling noise, and it was removed by using $\tilde{\Sigma}_\tau = \frac{1}{2}(\hat{\Sigma}_\tau + \hat{\Sigma}_\tau^\top)$.

(c) AMUSE, SOBI and Λ

AMUSE diagonalises $\tilde{\Sigma}_\tau$ for a single lag τ by an eigendecomposition and gives the orthogonal matrix U_τ. SOBI uses a set of lags $\mathcal{T}$ and finds the orthogonal matrix that diagonalises the matrices $\tilde{\Sigma}_\tau$ jointly (Nordhausen and Virta, 2018),

$$U_{\mathcal{T}}^\top = \underset{V^\top V = I_p}{\mathrm{argmax}} \sum_{\tau \in \mathcal{T}} \left\| \mathrm{diag}\left(V^\top \tilde{\Sigma}_\tau V\right) \right\|_F^2,$$

where $||\bullet||_F$ is the Frobenius norm.

In this report, SOBI used $\mathcal{T} = \{1, \ldots, 12\}$, and the joint diagonalisation was done with `rjd` from the package `JADE`. The diagonal elements of Λ are

$$\lambda_j^2 = \left(u_j^\top \tilde{\Sigma}_\tau u_j\right)^2 \ \text{(AMUSE)}, \qquad \lambda_j^2 = \sum_{\tau \in \mathcal{T}} \left(v_j^\top \tilde{\Sigma}_\tau v_j\right)^2 \ \text{(SOBI)},$$

where u_j and v_j are the columns of U_τ and $U_{\mathcal{T}}$, ordered so that $\lambda_0^2 \geq \cdots \geq \lambda_{p-1}^2$. These values are the inputs of $\varphi(k)$ in the ladle estimator, and they are non-zero only for components with serial dependence at some lag in $\mathcal{T}$.

1.2.2 Bootstrapping on serially dependent datasets

The iid bootstrap cannot be used for the SOS model, because it destroys the serial dependence that the estimators rely on. Let X be the $T \times p$ data matrix with rows $X_1, \ldots, X_T$. Since the AMUSE and SOBI algorithms are based on the autocovariance matrices of a whitened series, any resampling method must preserve the correlations between successive observations. If the rows are sampled independently, the resampled observations are uncorrelated at every lag. All lagged autocovariances of a bootstrap sample are then zero and the joint diagonaliser cannot be recovered (Nordhausen and Virta, 2018). Therefore, the bootstrap has to resample blocks of consecutive observations. This will keep the dependence within each block.

(a) Fixed-block bootstrap
The fixed-block bootstrap resamples blocks of B consecutive observations, so that the dependence within each block is preserved. Following Nordhausen and Virta (2018), a block length B is chosen and $h = \lceil T/B \rceil$ starting indices are drawn independently and uniformly,

$$I_1, \ldots, I_h \overset{iid}{\sim} \text{Unif}\{1, \ldots, T - B + 1\}.$$

The blocks $X_{I_j}, \ldots, X_{I_j+B-1}$ are concatenated in the order drawn, and the series is cut to length T if T is not a multiple of B. No wrapping is used. Observations near the ends of the sample therefore appear in fewer blocks than those in the middle, so the resampled series is not stationary (Politis and Romano, 1994).

(b) Stationary bootstrap
The stationary bootstrap of Politis and Romano (1994) resamples blocks of random length, so that the resampled series is stationary. Each block starts at an index s drawn uniformly from $\{1, \ldots, T\}$, and its length L is geometric with parameter $p_{sb} \in (0, 1]$,

$$P(L = m) = (1 - p_{sb})^{m-1} p_{sb}, \qquad m = 1, 2, \ldots, \qquad E(L) = \frac{1}{p_{sb}}.$$

The indices of a block are taken circularly, $X_{T+k} = X_k$, blocks are added until the series has length T, and the last block is cut. Conditional on the data, the resampled series is stationary, unlike the fixed-block bootstrap. In this report, the same block length ℓ was used for both methods, with $B = \ell$ for the fixed-block bootstrap and $p_{sb} = 1/\ell$ for the stationary bootstrap, so that the expected block length of the stationary bootstrap equals the block length of the fixed-block bootstrap (Nordhausen and Virta, 2018).

1.2.3 The ladle function and the ladle estimator

Finding the ladle estimator $\hat{q}$ starts by calculating the ladle function g. The ladle function is defined as follows: $g(k) = f(k) + \varphi(k)$, where $k = 0, \ldots, p-1$. The quantity p is the column dimension of the data. The question "How do you calculate the quantities f and φ then?" arises. Before obtaining f one needs to obtain f_0.

The function $f_0(k)$ is defined for $k = 1, \ldots, p-1$ as

$$\frac{1}{n} \sum_{j=1}^{n} (1 - |\det(\hat{B}_k^T B_{k,j}^*)|),$$

where $\hat{B}_k^T$ are the first k columns of the joint diagonaliser computed from the original sample and $B_{k,j}^*$ are the first k columns of the joint diagonaliser computed from the jth bootstrap sample. Additionally, we define $f_0(0) = 0$. The function $f(k)$ is then extracted from $f_0(k)$ by normalising in the following way:

$$f(k) = f_0(k) / \Big(1 + \sum_{l=0}^{p-1} f_0(l)\Big).$$

The function $\varphi(k)$ is defined with the diagonal elements of Λ, $\lambda_0^2, \lambda_1^2, \ldots, \lambda_{p-1}^2$, that are in a decreasing order. Λ has been obtained either by using either AMUSE or SOBI. The (normalised) φ function is now

$$\varphi(k) = \lambda_k^2 / \Big(1 + \sum_{l=0}^{p-1} \lambda_l^2\Big).$$

Now that all the ladle quantities f, φ and g have been defined, one can define the ladle estimator $\hat{q}$ as

$$\hat{q} = \underset{q=1,\ldots,p-1}{\arg\min} \big\{g(q)\big\}.$$

In the ladle function g, the information represented by the bootstrap-variability is found in the normalised variance function f. This is quite obvious, because it is the only quantity in g that contains components stemming from bootstrapping. The variance function f_0 is a way of finding out the average variance of the space, which is spanned by the first k vectors. The absolute determinant is a quantity, that takes values between 0 and 1, and it measures the similarity between $\hat{B}_k$ and $B^*_{k,j}$.

The information represented by the eigenvalues is found mainly in φ, but also in f. The function φ is essentially a scree plot, which contains the diagonal elements of the matrix Λ calculated with either AMUSE or SOBI. These values are seen as eigenvalues.

The bootstrap-variability stems from the eigenvectors that are being calculated for the matrix B^*_j, which is the joint diagonaliser computed from the jth bootstrap. As the order of magnitude of the eigenvalues in Λ decreases sharply, the bootstrap-variability of the eigenvectors will stay quite flat. When the bootstrap-variability of the eigenvectors increases sharply, the magnitude of the eigenvalues will also stay flat for the target matrix. However, the curve g will have a local minimum at the sweet spot, where both the eigenvalues and the bootstrap-variability have a negative jump.

1.2.4 Modelling of time series

In this task three different ways of modelling time series were used: the Autoregressive model AR(p), the Moving-average model MA(q), and the Autoregressive–moving-average model ARMA(p, q), where p and q are the lengths of the parameters used in the models. Here we will lay out the formulas used for each model.

The AR(p) model is defined as follows:

$$X_t = \sum_{i=1}^{p} \varphi_i X_{t-i} + \varepsilon_t,$$

where $\varphi = (\varphi_1, \ldots, \varphi_p)$ is the parameter of the model, and ε_t is standard normal distributed (white noise). Usually ε_t's are also independent from each other.

The MA(q) model is defined as:

$$X_t = \mu + \varepsilon_t + \sum_{i=1}^{q} \theta_i \varepsilon_{i-1},$$

where $\theta = (\theta_1, \ldots, \theta_q)$ is the parameter of the model, and $\varepsilon_1, \ldots, \varepsilon_t$ are Gaussian iid white noise. The parameter μ is the expectation of X_t, but it is often assumed to be 0.

Lastly, the ARMA(p, q) model is defined as:

$$X_t = \varepsilon_t + \sum_{i=1}^{p} \varphi_i X_{t-i} + \sum_{i=1}^{q} \theta_i \varepsilon_{t-i},$$

where the parameters are as described in AR(p) and MA(q) respectively.

1.3 Implementation and validation

The functions were written in `R` for a $T \times p$ data matrix with rows as time points, and each was checked against an independent computation. The same functions were then applied to the bootstrap samples.

1.3.1 Whitening, AMUSE and SOBI

The functions described below were written in `R`, and the names in parentheses are the names of these functions. The series is centred by the column means and whitened by $\hat{\Sigma}^{-1/2}$ (`center_series`, `whiten_series`). No additional scaling was used, because whitening with the full sample covariance matrix already removes differences in scale and contemporaneous correlation.

The matrices $\hat{\Sigma}_\tau$ are computed for lags $\tau \geq 1$ with the $1/T$ normalisation and without wrap-around, and are then symmetrised (`autocov_raw`, `symmetrize`). AMUSE (`amuse_fit`) uses `eigen` at lag 1, and SOBI (`sobi_fit`) uses `JADE::rjd` for lags $1, \ldots, 12$. Both return Λ in decreasing order.

The functions were checked on `timeseries.dat` ($T = 1000$, $p = 10$):

- **Whitening:** $\| \frac{1}{T} \sum_t \hat{x}_t^{st} \hat{x}_t^{st\top} - I_p \|_F = 9.7 \times 10^{-15}$.
- **Autocovariances:** identical to `acf()` at lags $1, \ldots, 12$ (maximum difference 0).
- **SOBI:** V is orthogonal ($\max |V^\top V - I_p| = 2.2 \times 10^{-15}$), and every component matches one component of `JADE::SOBI` with absolute correlation at least 0.9996 (all others at most 0.027).

1.3.2 Bootstrap functions and validation

Both bootstrap methods were implemented as index generators that return T row indices (`fixed_block_index`, `stationary_index`), so that whole rows are resampled and the dependence between the components is kept. The function `boot_samples` returns M bootstrap samples of size $T \times p$, which can be passed directly to `sobi_fit`. The geometric block lengths were drawn with `rgeom(1, p_sb) + 1` and the indices of the stationary bootstrap are taken circularly. A fixed seed was used, since the starting indices and block lengths are random.

The methods were tested on a bivariate VAR(1) series with $T = 1000$ and a lag-1 average autocorrelation of 0.697. For each method, $M = 200$ bootstrap samples were drawn with block length (or expected block length) $\ell = 20$. Figure 2 shows the autocorrelation function averaged over the components and the bootstrap samples. The iid bootstrap removes the dependence at all lags. The two block methods are close to each other and slightly below the original curve at the shorter lags, because the dependence is lost at the joins between blocks.

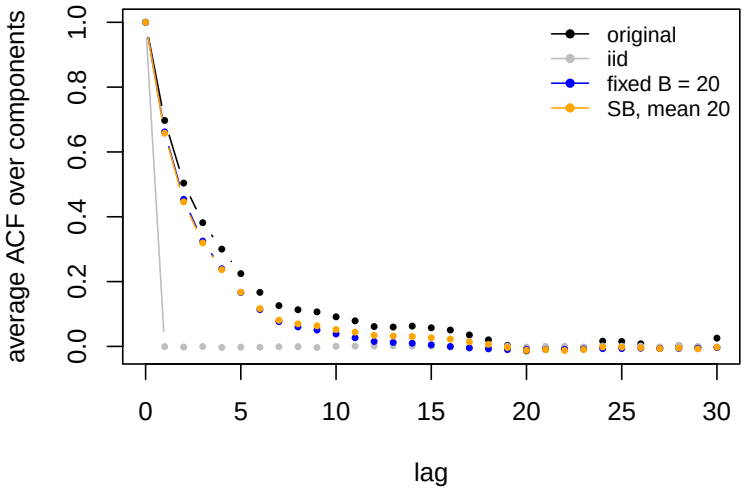


Figure 2: Average autocorrelation function of the VAR(1) series and of the bootstrap samples ($M = 200$, $\ell = 20$).

Table 1 shows the effect of the block length. At $\ell = 1$ both methods reduce to the iid bootstrap and the lag-1 autocorrelation is close to zero. As ℓ increases, this value approaches that of the original series, and the two methods give very similar values at every ℓ. The same functions also ran on `timeseries.dat`, giving bootstrap samples of size 1000×10, so they work for a general multivariate data matrix.

ℓ	Fixed-block	Stationary
1	−0.004	−0.003
5	0.555	0.551
10	0.624	0.622
20	0.659	0.661
50	0.683	0.679
100	0.691	0.685
200	0.698	0.690

Table 1: Average lag-1 autocorrelation of the bootstrap samples of the VAR(1) series for different block lengths ℓ ($M = 200$). The lag-1 autocorrelation of the original series is 0.697.

1.3.3 Validation of the custom `ladle` function

To calculate the ladle quantities, a custom function (`ladle`) was written in `R`. In order to validate that the function worked as desired, the function was tested on simulated data. The simulated data consisted of 1000 observations with four different variables: two signal components from a bivariate VAR(1) series, and two noise components from a bivariate standard normal distribution. The ladle quantities were then calculated for eight different function settings, where the diagonalisation method, bootstrap method and block length were varied. The number of bootstrap samples was 200.

Figure 3 shows plots of the ladle quantities. On the top row, SOBI was used for joint diagonalisation, and on the bottom row, AMUSE was the diagonaliser. The two first columns display plots where the block length

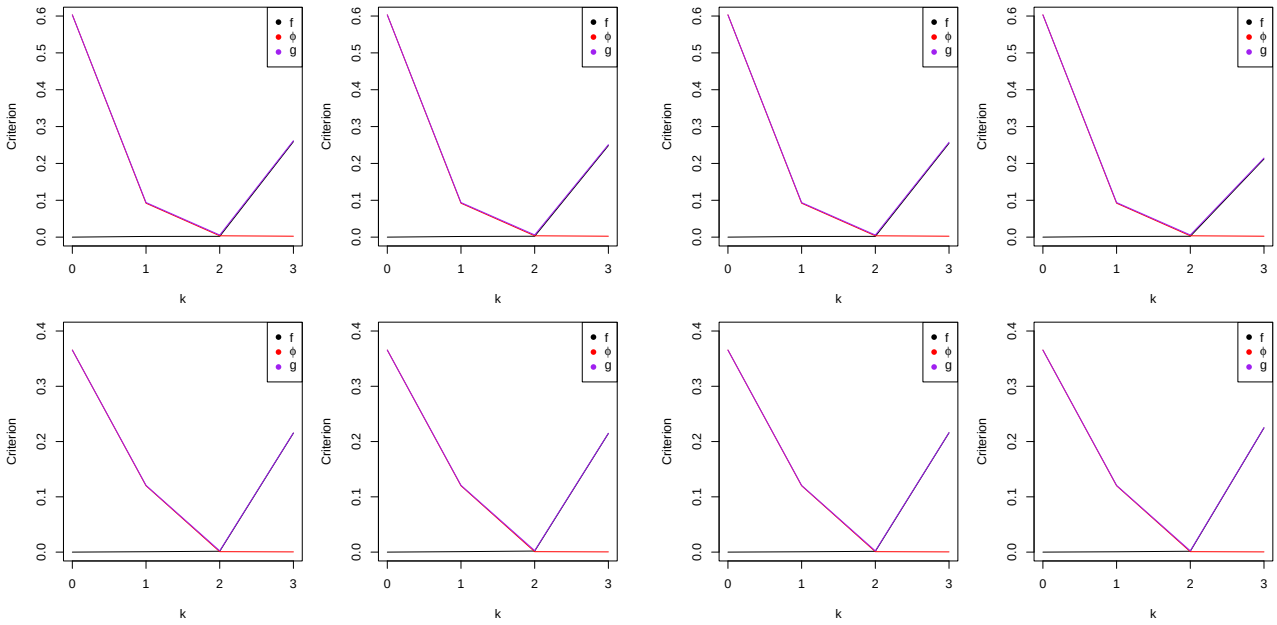


Figure 3: Ladle plots calculated for simulated data with eight different settings.

was 40, and in the two last columns the block length was 60. Regardless of setting, our implementation of the ladle function estimates the signal dimension $q = 2$ correctly. This indicates that our implementation indeed calculates the correct quantities, and is able to estimate the signal dimension.

1.4 Simulation study and analysis

In *Ladle Estimator for Time Series Signal Dimension* (Nordhausen and Virta, 2018), a simulation study was conducted to test, how well the ladle estimator can estimate the signal dimension. The simulation study was done on simulated data, which was described in the paper. In our simulation study we used the same data generating process described in the article.

The simulated data consists of multivariate data with dimension $p = 10$ and the signal dimension is $q = 5$, which means that there were five noise components. The signals were generated from the following models: ARMA(0.6; (0, -0.4)), ARMA((0.4, 0.1, 0.3); (0.2, 0.4)), AR(0.4), MA(0.5, 0.3, 0.1, 0.2) and AR(0.1, 0.1, 0.3), using custom functions in `R`. The five noise components were all Gaussian white noise, that is, the noise components all follow the distribution $N(0, 1)$. For the moving-average models it was assumed that $\mu = 0$ for X_t, where $t = 1, \ldots, 2000$. Further, Figure 4 shows a plot of the different components in the simulated data. The bare eye could not distinguish a real signal component from a noise component, which supports the need for using the ladle estimator.

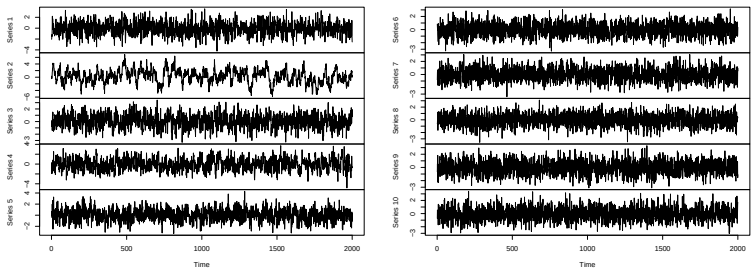


Figure 4: Time series plot of the simulated data.

To verify that the generated data actually consisted of five serially dependent components and five independent components, the autocorrelation function for each component was plotted. The plots can be found in Figure 5. For all the noise components autocorrelation stays below the threshold for meaningful autocorrelation for all lags. This concludes that the five noise components are white noise. All the signal components show autocorrelation at varying lags, as expected. The signal with the strongest autocorrelation is the one with the

most parameters, i.e., the ARMA((0.4, 0.1, 0.3); (0.2, 0.4)) signal, which shows meaningful autocorrelation up to 15 lags. For the other signal components the autocorrelation is not as strong, but it is still present.

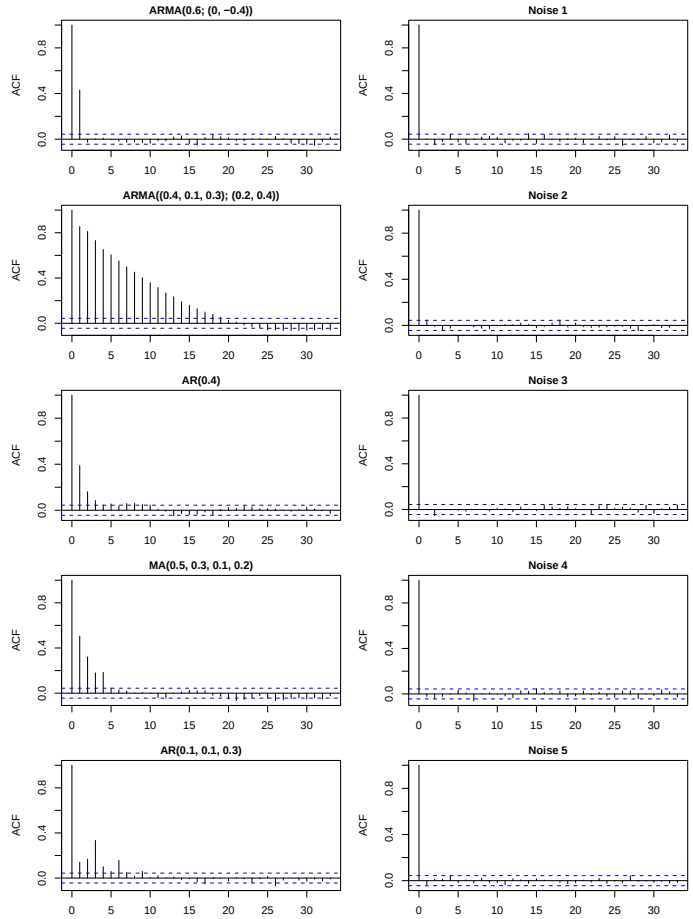


Figure 5: Autocorrelation function plots for simulated data. Signal components to the left and noise components to the right.

The simulation study itself was made by calculating the ladle estimator for 500 repetitions for varying signal lengths and bootstrap methods. For each repetition, the index, which contained the minimum value for the ladle function, was recorded. Signal duration lengths, that were considered, were $T = 200, 500, 1000, 2000$. For the bootstrap methods, different block lengths were used. For the stationary block bootstrap the expected block lengths were 20 and 40, and for the fixed block bootstrap the block lengths were 20 and 40 as well. These block lengths were partly chosen, because of the block lengths used in the referenced article. Another reason is what was established in subsection 1.3.2, that is, a longer block length allows us to mimic the average autocorrelation of the dataset. The chosen block lengths are over the threshold of mimicking the autocorrelation badly, but one of them could still perform worse than the other.

The ladle was computed for AMUSE with lag $\tau = 1$ and for SOBI with $\mathcal{T} = \{1, \ldots, 12\}$, just as in Nordhausen and Virta, 2018. The amount of bootstrap samples was low, only 10, so the computational cost would not be too high, and, because there were convergence issues for the `rjd()` function used in SOBI. The simulation took roughly 3 hours and for each joint diagonalisation, and for all bootstrap settings the individual simulation times were recorded. These times can be found in Table 2.

S_S_20	S_S_40	S_F_20	S_F_40	A_S_20	A_S_40	A_F_20	A_F_40
1.08 h	56.03 min	31.19 min	30.81 min	50.33 s	43.11 s	25.08 s	26.00 s

Table 2: Simulation times per setting.

In Table 2 the first letter refers to the diagonalisation method: ”S” for SOBI and ”A” for AMUSE. The second letter refers to the bootstrap method used: ”S” for stationary block and ”F” for fixed block. The number is what expected block length respectively block length was used. From this we can conclude that AMUSE is by far a faster joint diagonalisation method, and the fixed block bootstrap is the fastest bootstrap method. Using

a smaller block length or expected block length is also faster. However, this data does not tell us anything about the accuracy of the methods.

To check the Monte Carlo uncertainty and variability of the simulation we calculated the mean of the ladle function g for each repetition for every different setting. While this does not take into account the variability of an individual value of the function for each k, it allows us to look at the mean behaviour at each setting more effectively. The means were then plotted in separate boxplots, where one plot represents a single setting, and a box represents the means for different signal lengths. The boxplots are found in Figure 6.

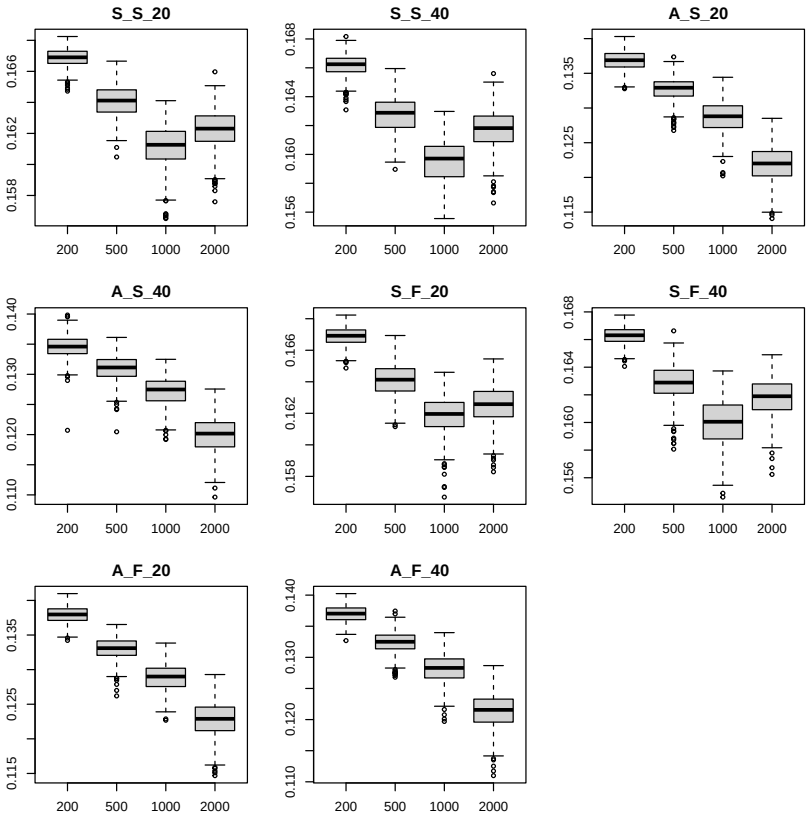


Figure 6: Boxplots that demonstrate the Monte Carlo variability and uncertainty of the bootstrap samples.

Judging from the plots in Figure 6, the Monte Carlo variability and uncertainty is higher when the signal length is higher. This is quite logical, because with a larger sample size, bootstrapping is made from a larger pool of values, and the probability of picking a certain value is lower. For SOBI the means range over a smaller interval with a length of about 0.012, whereas the means from AMUSE range over a larger interval of length between $0.025 - 0.030$. For the larger block length the interval length was larger. This means that SOBI gives more similar results, regardless of signal length, bootstrap method and block length, because the means vary over a smaller range overall, and the standard deviation is also smaller than for AMUSE.

We were not able to fully replicate the results in Nordhausen and Virta, 2018, but similar conclusions as were made in the article can be made. The plot results can be found in Figure 7. For $T = 200$, SOBI performed worse than AMUSE. The ladle estimate was either too high or too low for both bootstrap methods and both block lengths. However, a longer block length seems to make the calculated estimate closer to the real signal dimension. For both bootstrap methods, block length 40 has a lower proportion of 1's and 9's, than block length 20. For AMUSE the ladle estimate is 5 for a quite large proportion of the repetitions, regardless of method. The estimates also show a larger variety of values than for SOBI. For all bootstrap methods and block lengths the ladle estimator has been 4 most times.

For $T = 500$ AMUSE performs worse than SOBI. Now almost all ladle estimates for AMUSE have been 4, regardless of parameters. For stationary block bootstrap there is a tiny fraction of 5. For SOBI, the results vary, but the majority of ladle estimates were 5, which is the correct value. When $T = 1000$, SOBI is the best method as well. In our simulation AMUSE led to the ladle estimator 4 for all methods, whereas SOBI "guessed" right every time. Almost the same thing can be said about $T = 2000$. The only difference is that AMUSE now has a few right minimums for the ladle function, but the vast majority is still 4.

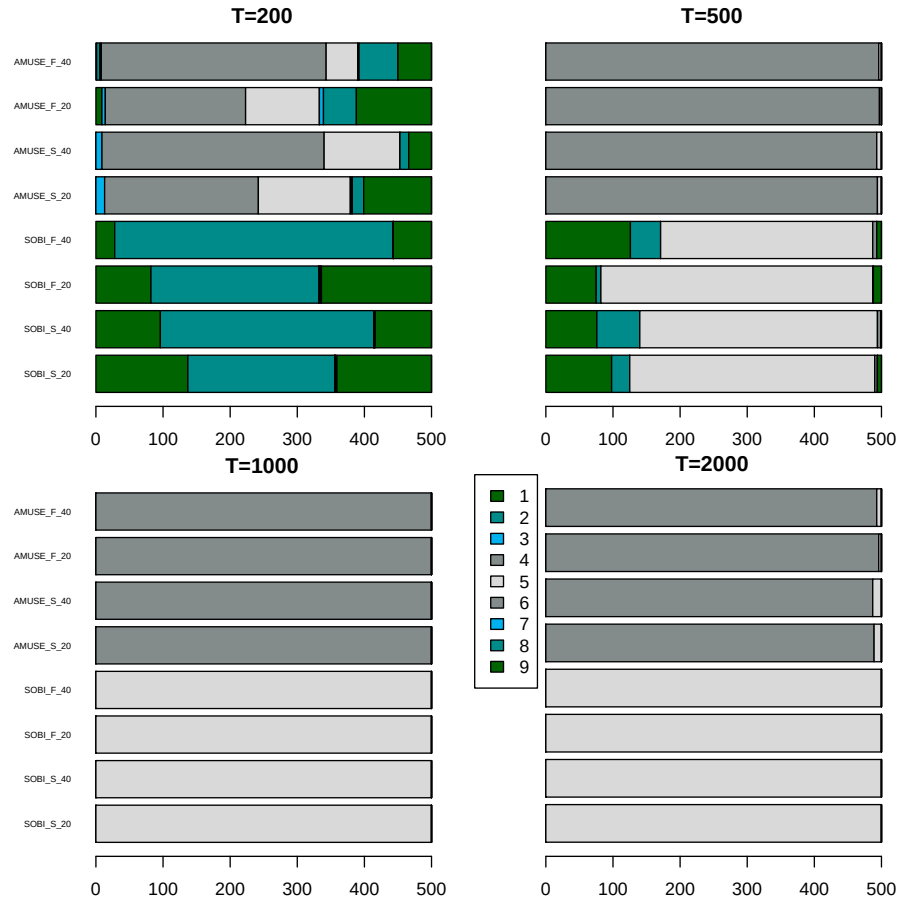


Figure 7: The distributions of the ladle estimators.

From the simulation study we can conclude that using SOBI for joint diagonalisation is the best option for a larger dataset. For $T = 1000$ or $T = 2000$, it does not matter which bootstrap method you use and block length does not matter either, given that SOBI is used. For $T = 500$, SOBI is preferred as well, and fixed block bootstrap with length seems like the best option. For a smaller dataset, one would maybe prefer AMUSE with block length or expected block length 40.

1.5 Application to `timeseries.dat`

In subsection 1.4 conclusions were made about which joint diagonalisation and bootstrap method should be used when. As mentioned in the problem statement, the `timeseries.dat` dataset contains 1000 observations, so, based on the simulation study the best option is to use SOBI for joint diagonalisation. For the estimation of the ladle quantities, 100 bootstrap samples were used for all settings, when applying the custom function to the provided dataset.

Four different settings were used to calculate the ladle quantities:

(1) Fixed block bootstrap with block length 20;

(2) Fixed block bootstrap with block length 40;

(3) Stationary block bootstrap with expected block length 20;

(4) Stationary block bootstrap with expected block length 40.

The block lengths were chosen for consistency, when compared to the simulation study, and, because a longer block length mimics the autocorrelation of the dataset better.

Figure 8 shows plots of the different ladle quantities for the four different settings, where joint diagonalisation was done with SOBI. For all different settings, the ladle function g behaves quite similarly, that is, the general shape of g is the same. The values on the y-axis also match, so all settings give roughly the same estimated values. From this also follows, that all settings have the same ladle estimator: $\hat{q} = 3$. So, the estimated true signal dimension for the dataset is 3.

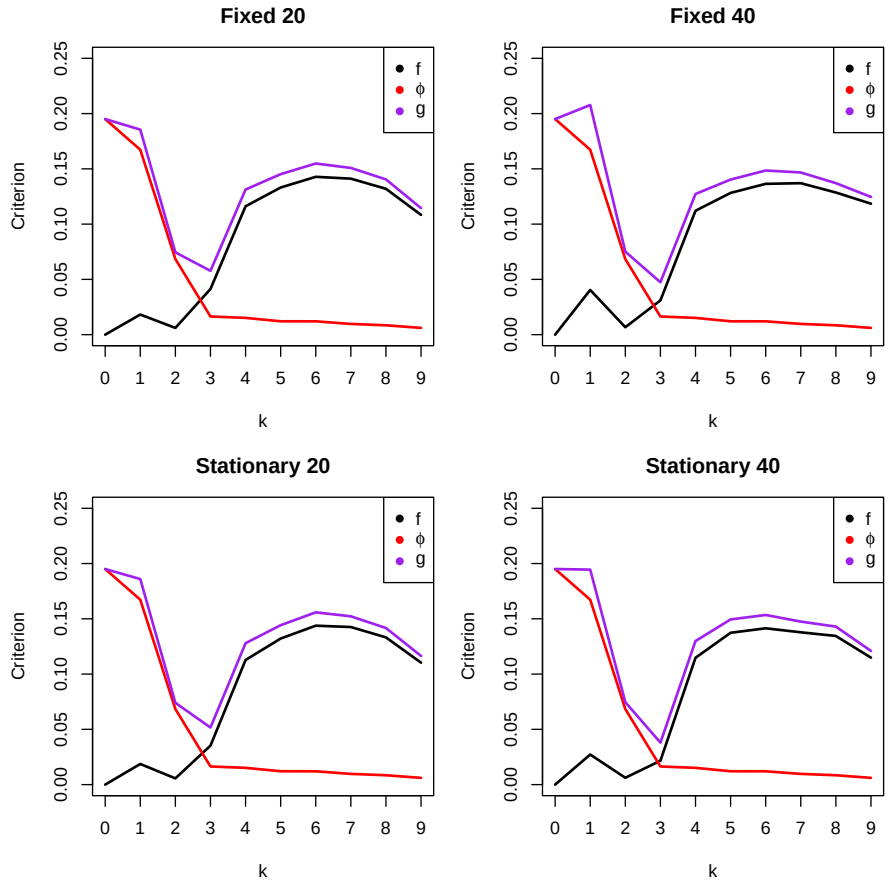


Figure 8: Ladle plots for `timeseries.dat`.

1.6 Conclusions

In this task we have investigated the use of bootstrap methods aimed specifically at serially dependent data, in order to determine the signal dimension q of a time series dataset with the help of the ladle estimator $\hat{q}$. To calculate the ladle estimator we utilised a diagonaliser of the series, and the eigenvalue quantity Λ. Joint diagonalisation was done with either AMUSE or SOBI.

In order to determine which bootstrap method, block length for the bootstrapping and diagonaliser to use, a simulation study was done. What can be concluded from this simulation study is, that SOBI gives a more accurate estimate of the ladle quantity for signal lengths higher than $T = 500$, whereas AMUSE could be preferred for a smaller sample size. For larger sample sizes ($T = 1000$ or $T = 2000$), block length does not seem to influence the results, when SOBI is used.

The results from the simulation study influenced the choice of joint diagonalisation method in the application of the ladle function on `timeseries.dat`. SOBI was used for joint diagonalisation, but four different settings of varying bootstrap method and block length were tested. All four settings gave ladle quantities with similar shapes, so, they all agreed on one common ladle estimator $\hat{q}$. The signal was estimated to be 3, which implies that among the 10 signals in the time series, 7 of the signals are noise. The methods used here do not, however, help us identify, which variables are signals and which are noise.

2 Bayesian Mixture Modelling with MCMC

Gaussian mixture models are used to describe data containing multiple groups. However, estimating the model parameters can be difficult when the groups overlap or when label switching occurs. In this problem, we implement a Gibbs sampler with MCMC method, to estimate the parameters of a two-component Gaussian mixture model.

2.1 The Bayesian model

In parametric statistical inference, the observed data y are assumed to arises from a probability distribution of an unknown parameter vector θ. The two main approaches are frequentist and Bayesian inference. Frequentist inference treats parameters as fixed, but unknown, whereas Bayesian inference represents uncertainty about the parameters through probability distributions.

In Bayesian inference, the prior distribution is combined with the observed data through the likelihood function to obtain the posterior distribution using Bayes' rule:

$$f_{\Theta|Y}(\theta \mid y) = \frac{f_{Y|\Theta}(y \mid \theta) f_{\Theta}(\theta)}{f_Y(y)},$$

where $f_{\Theta|Y}(\theta \mid y)$ is the posterior density, $f_{Y|\Theta}(y \mid \theta)$ is the likelihood, $f_{\Theta}(\theta)$ is the prior density, and $f_Y(y)$ is the marginal likelihood. Since the marginal likelihood does not depend on θ, Bayes' rule can also be expressed in proportional form:

$$f_{\Theta|Y}(\theta \mid y) \propto f_{Y|\Theta}(y \mid \theta) f_{\Theta}(\theta)$$

Therefore, the main objective of Bayesian inference is to obtain the posterior distribution, which combines prior information with evidence from the observed data to describe uncertainty about the unknown parameters.

2.2 Bayesian mixture model and latent variable

A Gaussian mixture model is a statistical model that assumes the observed data arise from a combination of two or more normal distributions rather than a single distribution, and each component has its own mean and variance. A Bayesian Gaussian mixture model uses Bayesian statistical methods to estimate the unknown parameters and quantify the uncertainty associated with them.

In a mixture model, a latent variable is introduced to represent the component membership of each observation. A latent variable is a variable that cannot be directly observed, but is assumed to exist within the statistical model. For example, in a two-component Gaussian mixture model, the latent variable Z_i indicates, whether observation y_i belongs to the first or second component. Since the component is unknown, Bayesian inference allows us to estimate the mixing probability and the latent component allocations using the observed data.

2.2.1 Model Specification

In this problem, we consider a two-component Gaussian mixture model with a common variance and two different means. Let $y = (y_1, \ldots, y_n)$ denote the observed data. The probability density function of each observation is

$$f(y_i \mid \theta) = \pi \varphi(y_i; \mu_1, \sigma^2) + (1 - \pi)\varphi(y_i; \mu_2, \sigma^2),$$

where $\varphi(y_i; \mu, \sigma^2)$ denotes the density of a normal distribution with mean μ and variance σ^2. The unknown parameter vector is

$$\theta = (\mu_1, \mu_2, \pi, \sigma^2)$$

where μ_1 and μ_2 are the means of the two components, $\sigma^2 > 0$ is their common variance, and $0 < \pi < 1$ is the probability that an observation belongs to the first component. Further, $1 - \pi$ is the probability of belonging to the second component.

To identify which component generates each observation, we introduce a latent variable Z_i, defined as

$$Z_i = \begin{cases} 1, & \text{if observation } i \text{ belongs to component 1,} \\ 2, & \text{if observation } i \text{ belongs to component 2.} \end{cases}$$

Where:

$$P(Z_i = 1 \mid \pi) = \pi, \qquad P(Z_i = 2 \mid \pi) = 1 - \pi \tag{1}$$

Conditional on the component allocation, the observations follow normal distributions:

$$Y_i \mid Z_i, \mu_1, \mu_2, \sigma^2 \sim \begin{cases} N(\mu_1, \sigma^2), & Z_i = 1, \\ N(\mu_2, \sigma^2), & Z_i = 2. \end{cases}$$

Assuming the observations are conditionally independent given the model parameters, the likelihood function is

$$L(\theta; y) = \prod_{i=1}^{n} \left[\pi\varphi(y_i; \mu_1, \sigma^2) + (1-\pi)\varphi(y_i; \mu_2, \sigma^2)\right].$$

By introducing the latent allocations $Z = (Z_1, \ldots, Z_n)$, the complete-data likelihood can be written as

$$P(y \mid \theta) = L(\theta; y, Z) = \prod_{i=1}^{n} \left[\pi\varphi(y_i; \mu_1, \sigma^2)\right]^{(Z_i=1)} \times \left[(1-\pi)\varphi(y_i; \mu_2, \sigma^2)\right]^{(Z_i=2)}, \tag{2}$$

2.2.2 Prior Distributions

In Bayesian inference, prior distributions describe uncertainty about the unknown parameters before observing the data. In this two-component Gaussian mixture model, the unknown parameters are μ_1, μ_2, π, and σ^2.

(a) Prior μ_1 and μ_2
According to the problem, we assume that the two component means are independently normally distributed:

$$\mu_1, \mu_2 \overset{\text{ind}}{\sim} N(m_0, s_0^2)$$

The standardised observations have a sample mean of 0 and a sample variance of 1. So we choose $m_0 = 0$, because the observations are centred around zero after standardisation.

As for the hyperparameters; the prior standard deviation s_0 has to be chosen based on how widely we think the component means can vary around m_0. For example, suppose we believe that the component means lie within the interval $[-10, 10]$. Then we can choose $s_0 = 5$, which gives the prior distribution $\mu_1, \mu_2 \overset{\text{ind}}{\sim} N(0, 5^2)$.

For a normal distribution, the approximate 95% prior interval is

$$[m_0 - 1.96s_0,\ m_0 + 1.96s_0] = [0 - 1.96(5),\ 0 + 1.96(5)] = [-9.80,\ 9.80]$$

Therefore, this prior assigns approximately 95% probability to each component mean being between -9.80 and 9.80.

(b) Prior π
According to the problem, we assume the mixing probability has a Beta distribution:

$$\pi \sim \text{Beta}(a, b)$$

We know that $0 < \pi < 1$, therefore, all values have to be between 0 and 1. If the prior distribution is Beta, then the obvious choice for the hyperparameters are $a = 1$ and $b = 1$, which is Beta$(1, 1)$, a uniform prior on [0, 1].

(c) Prior σ^2
The common variance is assumed to have an inverse-Gamma prior distribution:

$$\sigma^2 \sim \text{InvGamma}(\alpha_0, \beta_0)$$

Because the standardised data has overall variance 1, by taking $\sigma^2 \sim \text{InvGamma}(2, 1)$, which means $\alpha_0 = 2$ and $\beta_0 = 1$, gives $E(\sigma^2) = 1$

$$E(\sigma^2) = \frac{\beta_0}{\alpha_0 - 1} = \frac{1}{2-1} = 1$$

Also, with shape 2, the inverse-Gamma distribution has substantial uncertainty, including a heavy right tail.

In summary, the prior distributions are

$$\begin{aligned} \mu_1 &\sim N(0, 25) \\ \mu_2 &\sim N(0, 25) \\ \pi &\sim \text{Beta}(1, 1) \\ \sigma^2 &\sim \text{InvGamma}(2, 1) \end{aligned}$$

The priors are assumed to be mutually independent. Therefore, the joint prior distribution can be written as:

$$p(\theta) = p(\mu_1)p(\mu_2)p(\pi)p(\sigma^2) \tag{3}$$

Therefore, using Bayes' theorem, the joint posterior distribution:

$$p(\theta, Z \mid y) = \frac{p(y \mid \theta, Z)p(\theta, Z)}{p(y)}$$
$$p(\theta, Z \mid y) \propto p(y \mid \theta, Z)p(\theta, Z)$$

Since the joint prior distribution can be written as

$$p(\theta, Z) = p(Z \mid \pi)p(\theta),$$

the posterior distribution becomes:

$$p(\theta, Z \mid y) \propto p(y \mid \theta, Z)p(Z \mid \pi)p(\theta)$$

Finally, using the independent prior distributions defined in Equation (3):

$$p(\theta, Z \mid y) \propto p(y \mid \theta, Z)p(Z \mid \pi)p(\mu_1)p(\mu_2)p(\pi)p(\sigma^2)$$

Since deriving and directly sampling from this joint posterior distribution is analytically difficult, a Gibbs sampler is used to generate samples, by updating the unknown parameters and latent component allocations from their full conditional distributions.

2.3 Derivation of the Gibbs Sampler

The conditional Distribution of Z_i, μ_1, μ_2, π, and σ^2 are derived below.

2.3.1 Conditional Distribution of Z_i

The objective is to find the full conditional distribution of Z_i, given y_i, μ_1, μ_2, π, and σ^2.

The two-component Gaussian mixture model is

$$Y_i \mid Z_i = k \sim N(\mu_k, \sigma^2), \qquad k \in \{1, 2\}$$

Therefore, let's take component 1,

$$Y_i \mid Z_i = 1 \sim N(\mu_1, \sigma^2)$$
$$P(Z_i = 1) = \pi$$

and for component 2,

$$Y_i \mid Z_i = 2 \sim N(\mu_2, \sigma^2)$$
$$P(Z_i = 2) = 1 - \pi$$

Using the multiplication rule,

$$f(y_i, z_i) = f(y_i \mid z_i)P(Z_i = z_i)$$

if $Z_i = 1$:

$$f(y_i, z_i = 1) = \pi f_1(y_i) \tag{4}$$

if $Z_i = 2$:

$$f(y_i, z_i = 2) = (1 - \pi)f_2(y_i) \tag{5}$$

For n observations, let n_1 denote the number of observations allocated to component 1 and n_2 denote the number allocated to component 2. Then,

$$n_1 + n_2 = n$$

So, the complete likelihood

$$f(y, z) = \pi^{n_1}(1 - \pi)^{n_2} \prod_{i:z_i=1} f_1(y_i) \prod_{i:z_i=2} f_2(y_i)$$

From Bayes' theorem,

$$P(Z_i = 1 \mid y_i) = \frac{f(y_i, Z_i = 1)}{f(y_i)} \tag{6}$$

Total probability,

$$\begin{aligned} f(y_i) &= f(y_i, Z_i = 1) + f(y_i, Z_i = 2) \\ &= \pi f_1(y_i) + (1 - \pi) f_2(y_i) \end{aligned} \tag{7}$$

Substituting 4 and 7 into 6,

$$\boxed{P(Z_i = 1 \mid y_i) = \frac{\pi f_1(y_i)}{\pi f_1(y_i) + (1 - \pi) f_2(y_i)}}$$

Similarly,

$$\boxed{P(Z_i = 2 \mid y_i) = \frac{(1 - \pi) f_2(y_i)}{\pi f_1(y_i) + (1 - \pi) f_2(y_i)}}$$

2.3.2 Conditional Distribution of μ_1 and μ_2

First, consider the conditional distribution of μ_1. The objective is to find

$$f(\mu_1 \mid y, z, \sigma^2)$$

The prior distribution is

$$\mu_1 \sim N(m_0, s_0^2)$$

From Bayes' theorem,

$$f(\mu_1 \mid y, z, \sigma^2) \propto f(y \mid z, \mu_1, \sigma^2) f(\mu_1) \tag{8}$$

The normal prior density is

$$f(\mu_1) = \frac{1}{\sqrt{2\pi s_0^2}} \exp\left(-\frac{(\mu_1 - m_0)^2}{2 s_0^2} \right)$$

Ignoring constants that do not depend on μ_1,

$$f(\mu_1) \propto \exp\left(-\frac{(\mu_1 - m_0)^2}{2 s_0^2} \right) \tag{9}$$

The likelihood contains only observations allocated to component 1. Therefore,

$$f(y \mid z, \mu_1, \sigma^2) \propto \prod_{i:z_i=1} \exp\left(-\frac{(y_i - \mu_1)^2}{2\sigma^2} \right)$$

$$f(y \mid z, \mu_1, \sigma^2) \propto \exp\left(-\frac{\sum_{i:z_i=1} (y_i - \mu_1)^2}{2\sigma^2} \right) \tag{10}$$

Substituting 9 and 10 into 8,

$$f(\mu_1 \mid y, z, \sigma^2) \propto \exp\left(-\frac{\sum_{i:z_i=1} (y_i - \mu_1)^2}{2\sigma^2} \right) \times \exp\left(-\frac{(\mu_1 - m_0)^2}{2 s_0^2} \right)$$

Combining the exponential terms,

$$f(\mu_1 \mid y, z, \sigma^2) \propto \exp\left[-\frac{1}{2} \left(\frac{\sum_{i:z_i=1} (y_i - \mu_1)^2}{\sigma^2} + \frac{(\mu_1 - m_0)^2}{s_0^2} \right) \right]$$

Expanding the squared terms,

$$\sum_{i:z_i=1} (y_i - \mu_1)^2 = \sum_{i:z_i=1} y_i^2 - 2\mu_1 \sum_{i:z_i=1} y_i + n_1 \mu_1^2$$

and

$$(\mu_1 - m_0)^2 = \mu_1^2 - 2\mu_1 m_0 + m_0^2$$

Removing terms that do not depend on μ_1,

$$f(\mu_1 \mid y, z, \sigma^2) \propto \exp\left[-\left(\frac{n_1}{2\sigma^2} + \frac{1}{2 s_0^2} \right) \mu_1^2 + \left(\frac{\sum_{i:z_i=1} y_i}{\sigma^2} + \frac{m_0}{s_0^2} \right) \mu_1 \right] \tag{11}$$

Normal density can be written as

$$f(x) \propto \exp\left(-\frac{(x-m)^2}{2v}\right)$$
$$\propto \exp\left(-\frac{x^2 - 2xm + m^2}{2v}\right)$$
$$f(x) \propto \exp\left(-\frac{x^2}{2v} + \frac{m}{v}x\right) \tag{12}$$

By comparing 11 and 12, we obtain

$$\frac{1}{2v_1} = \frac{n_1}{2\sigma^2} + \frac{1}{2s_0^2}$$

Therefore,

$$\boxed{v_1 = \left(\frac{n_1}{\sigma^2} + \frac{1}{s_0^2}\right)^{-1}}$$

Similarly,

$$\frac{m_1}{v_1} = \frac{\sum_{i:z_i=1} y_i}{\sigma^2} + \frac{m_0}{s_0^2}$$

Therefore,

$$\boxed{m_1 = v_1\left(\frac{\sum_{i:z_i=1} y_i}{\sigma^2} + \frac{m_0}{s_0^2}\right)}$$

The conditional distribution for μ_1 is:

$$\boxed{\mu_1 \mid y, z, \sigma^2 \sim N(m_1, v_1)}$$

Similarly, for μ_2:

$$\boxed{\mu_2 \mid y, z, \sigma^2 \sim N(m_2, v_2)}$$

where

$$\boxed{v_2 = \left(\frac{n_2}{\sigma^2} + \frac{1}{s_0^2}\right)^{-1}}$$

and

$$\boxed{m_2 = v_2\left(\frac{\sum_{i:z_i=2} y_i}{\sigma^2} + \frac{m_0}{s_0^2}\right)}$$

2.3.3 Conditional Distribution of π

The objective is to find

$$f(\pi \mid z)$$

We know that

$$P(Z_i = 1) = \pi \qquad P(Z_i = 2) = 1 - \pi$$

The prior distribution for π is

$$\pi \sim \text{Beta}(a, b)$$

Beta density:

$$f(\pi) \propto \pi^{a-1}(1-\pi)^{b-1} \tag{13}$$

The likelihood is then

$$f(z \mid \pi) = \pi^{n_1}(1-\pi)^{n_2} \tag{14}$$

From Bayes' theorem,

$$f(\pi \mid z) \propto f(z \mid \pi) f(\pi).$$

Substituting 13 and 14,

$$\begin{aligned} f(\pi \mid z) &\propto \pi^{n_1}(1-\pi)^{n_2} \times \pi^{a-1}(1-\pi)^{b-1} \\ &\propto \pi^{n_1+a-1}(1-\pi)^{n_2+b-1} \end{aligned}$$

Therefore,

$$\boxed{\pi \mid z \sim \text{Beta}(a+n_1, b+n_2)}$$

2.3.4 Conditional Distribution of σ^2

The objective is the conditional distribution of the variance σ^2

The prior distribution is:

$$\sigma^2 \sim \text{InvGamma}(\alpha_0, \beta_0),$$

where inverse-Gamma density is

$$f(\sigma^2) \propto (\sigma^2)^{-(\alpha_0+1)} \exp\left(-\frac{\beta_0}{\sigma^2}\right) \tag{15}$$

The likelihood function is now

$$f(y \mid z, \mu_1, \mu_2, \sigma^2) \propto (\sigma^2)^{-n/2} \exp\left[-\frac{\sum_{i=1}^{n}(y_i - \mu_{z_i})^2}{2\sigma^2}\right], \tag{16}$$

where

$$\mu_{z_i} = \begin{cases} \mu_1, & z_i = 1 \\ \mu_2, & z_i = 2 \end{cases}$$

From Bayes' theorem,

$$f(\sigma^2 \mid y, z, \mu_1, \mu_2) \propto f(y \mid z, \mu_1, \mu_2, \sigma^2) f(\sigma^2)$$

Substituting 15 and 16,

$$f(\sigma^2 \mid \cdots) \propto (\sigma^2)^{-n/2} \exp\left[-\frac{\sum_{i=1}^{n}(y_i - \mu_{z_i})^2}{2\sigma^2}\right] \times (\sigma^2)^{-(\alpha_0+1)} \exp\left(-\frac{\beta_0}{\sigma^2}\right)$$

Combining the powers and exponential terms,

$$f(\sigma^2 \mid \cdots) \propto (\sigma^2)^{-(\alpha_0+n/2+1)} \times \exp\left[-\frac{\beta_0 + \frac{1}{2}\sum_{i=1}^{n}(y_i - \mu_{z_i})^2}{\sigma^2}\right]$$

This is the kernel of an inverse-Gamma distribution. Therefore,

$$\boxed{\sigma^2 \mid y, z, \mu_1, \mu_2 \sim \text{InvGamma}\left(\alpha_0 + \frac{n}{2}, \beta_0 + \frac{1}{2}\sum_{i=1}^{n}(y_i - \mu_{z_i})^2\right)}$$

2.4 Gibbs Sampler, Mixture, and MCMC

2.4.1 Gibbs Sampler

Based on the conditional distributions that we derived in the previous section, a Gibbs sampler was implemented in `R` under the `gibbs_sampler` function. The purpose of the Gibbs sampler is to generate samples from the joint posterior distribution of the unknown parameters and latent component allocations. The unknown parameters are $\theta = (\mu_1, \mu_2, \pi, \sigma^2)$, while $Z = (Z_1, \ldots, Z_n)$ represents the latent variables.

The algorithm starts by assigning initial values that we discussed in section 2.2.2 to the unknown parameters. Then, step by step, the Gibbs sampler updates the unknown parameter as below:

(1) Update the latent variable Z_i.

(2) Update μ_1 by sampling from its conditional normal distribution.

(3) Update μ_2 by sampling from its conditional normal distribution.

(4) Update the mixing probability π by sampling from its conditional Beta distribution.

(5) Update the common variance σ^2 by sampling from its conditional inverse-Gamma distribution.

As for the latent variable, the conditional probabilities are calculated using the log density function, because it improves numerical stability, by avoiding direct calculations involving very small probabilities. The resulting posterior samples of μ_1, μ_2, π, and σ^2 are stored in a data frame, while the latent allocations are stored separately in a matrix.

2.4.2 Mixture

To check the performance of the Gibbs sampler, we generated two sets of two-component Gaussian mixture model. For both simulations, the sample size was set to $n = 200$, the common variance was set to $\sigma^2 = 1$, and the mixing probability was set to $\pi = 0.5$.

The two simulations were:

(1) Clearly separated components: $\mu_1 = -3$ and $\mu_2 = 3$.

(2) Substantially overlapping components: $\mu_1 = -0.5$ and $\mu_2 = 0.5$.

The datasets were generated using the `simulate_mixture` function in `R`. First, the component allocations were generated according to the mixing probability. Then, each observation was generated from the normal distribution corresponding to its component.

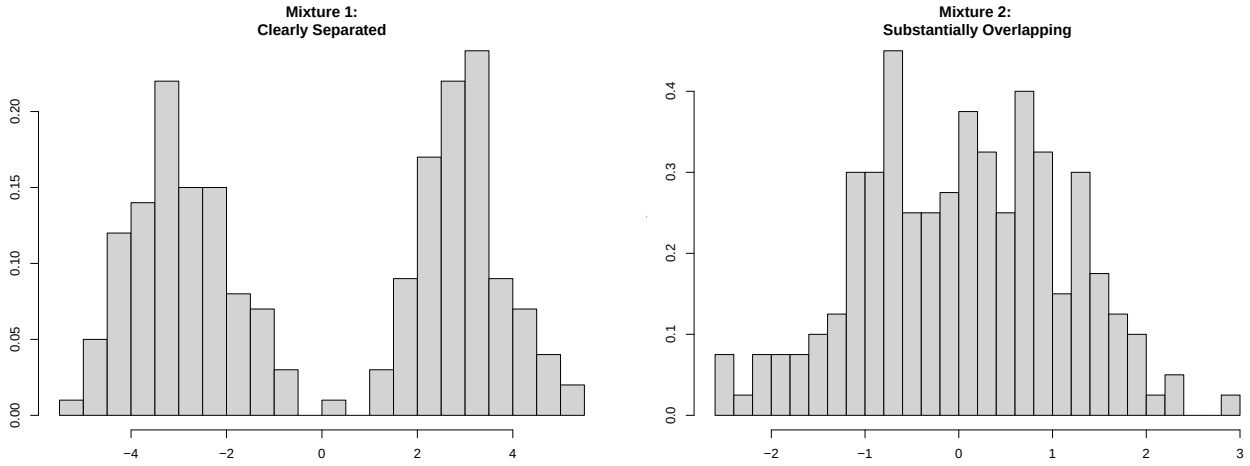


Figure 9: Histograms of the two simulated Gaussian mixtures: clearly separated components (left) and substantially overlapping components (right).

(a) Clearly Separated Mixture

For the first simulation, the model parameters were

$$\mu_1 = -3, \qquad \mu_2 = 3, \qquad \sigma^2 = 1, \qquad \pi = 0.5.$$

Since the difference between the component means is 6, the two component means are separated, and the simulation shows two clear peaks. So, observations close to -3 are more likely to originate from the first component, while observations close to 3 are more likely to originate from the second component. Therefore, the observed data seems to provide strong information about the component allocations.

(b) Substantially Overlapping Mixture

For the second simulation, the parameters were

$$\mu_1 = -0.5, \qquad \mu_2 = 0.5, \qquad \sigma^2 = 1, \qquad \pi = 0.5.$$

In this case, the difference between the component means is only one unit, and both components have standard deviation one. So, their distributions overlap substantially. Further, the resulting mixture does not show two clearly separated peaks. Therefore, the observed data provide less information about the component allocations than in the clearly separated case.

Overall, in the clearly separated mixture, an observation of -3 is very likely to belong to component 1, while $+3$ is very likely to belong to component 2. However, in the overlapping mixture, an observation of 0 is equally likely to have come from either component. Therefore, the substantial overlap case seems to be the more difficult MCMC mixing.

2.4.3 MCMC

To investigate the behaviour of the Gibbs sampler, four chains were generated for each simulated dataset using different initial component means:

$$\begin{aligned}
&\text{Chain 1: } (-0.5, 0.5),\\
&\text{Chain 2: } (0.5, -0.5),\\
&\text{Chain 3: } (-3, 3),\\
&\text{Chain 4: } (3, -3).
\end{aligned}$$

For all chains, the initial mixing probability and common variance were set to

$$\pi^{(0)} = 0.5, \qquad (\sigma^2)^{(0)} = 1.$$

Each chain was run for 12 000 stored states, including the initial state. The first 2000 states were discarded as burn-in, leaving 10 000 posterior draws per chain. In particular, the initial values include both possible arrangements of the component means, so that it would be useful for investigating label switching. Below are the MCMC trace plots for the clearly separated and substantially overlapping mixtures for chain 1.

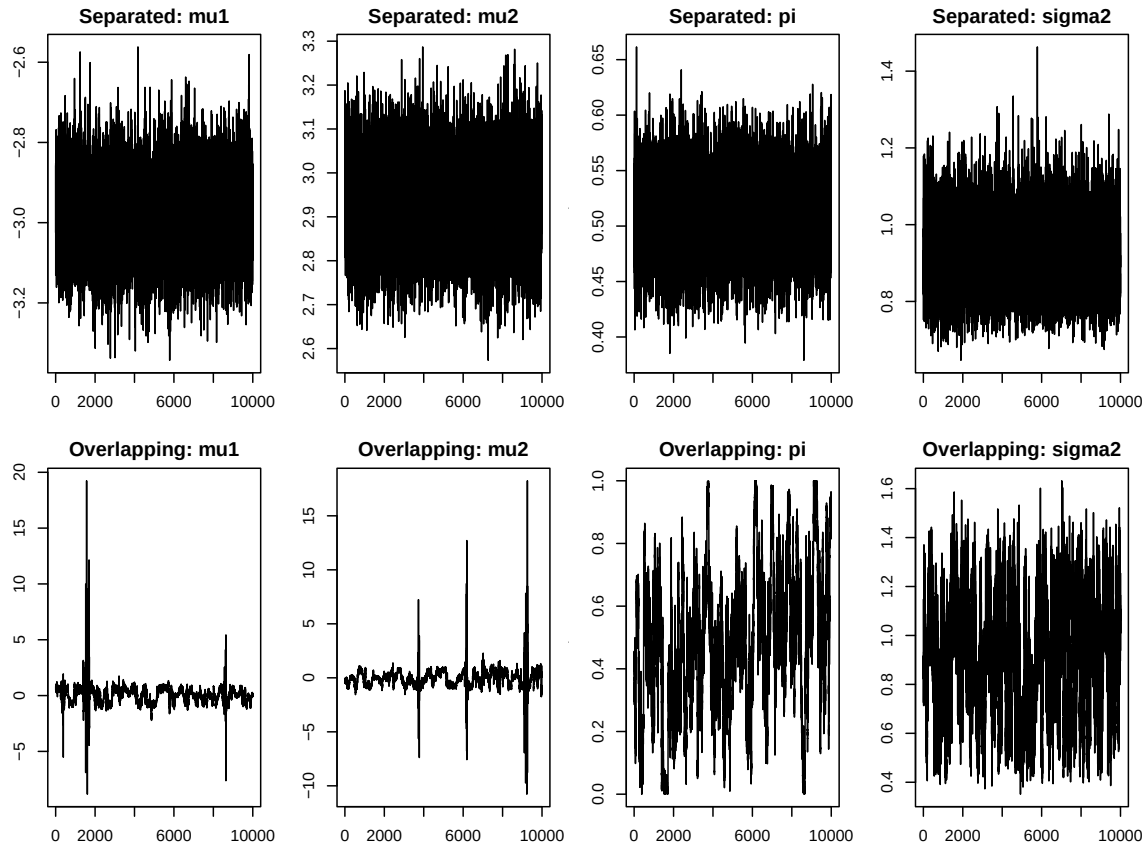


Figure 10: MCMC trace plots for the clearly separated (top row) and substantially overlapping (bottom row) Gaussian mixtures. The plots show the posterior samples of μ_1, μ_2, π, and σ^2 of iterations of the Gibbs sampler.

Figure 10 represents the MCMC trace plots for the clearly separated and substantially overlapping Gaussian mixtures. In the clearly separated mixture, the chains appear to fluctuate around relatively stable values for all four parameters. But the overlapping mixture shows a greater variation, particularly in the component means and mixing probability. The component means occasionally show large movements while, overall, staying relatively close to zero, while the mixing probability fluctuates over a much wider range compared to the clearly separated case. These observations suggest that the Gibbs sampler has greater difficulty estimating the parameters, when the two components overlap substantially.

Clearly Separated Mixture **Substantially Overlapping Mixture**

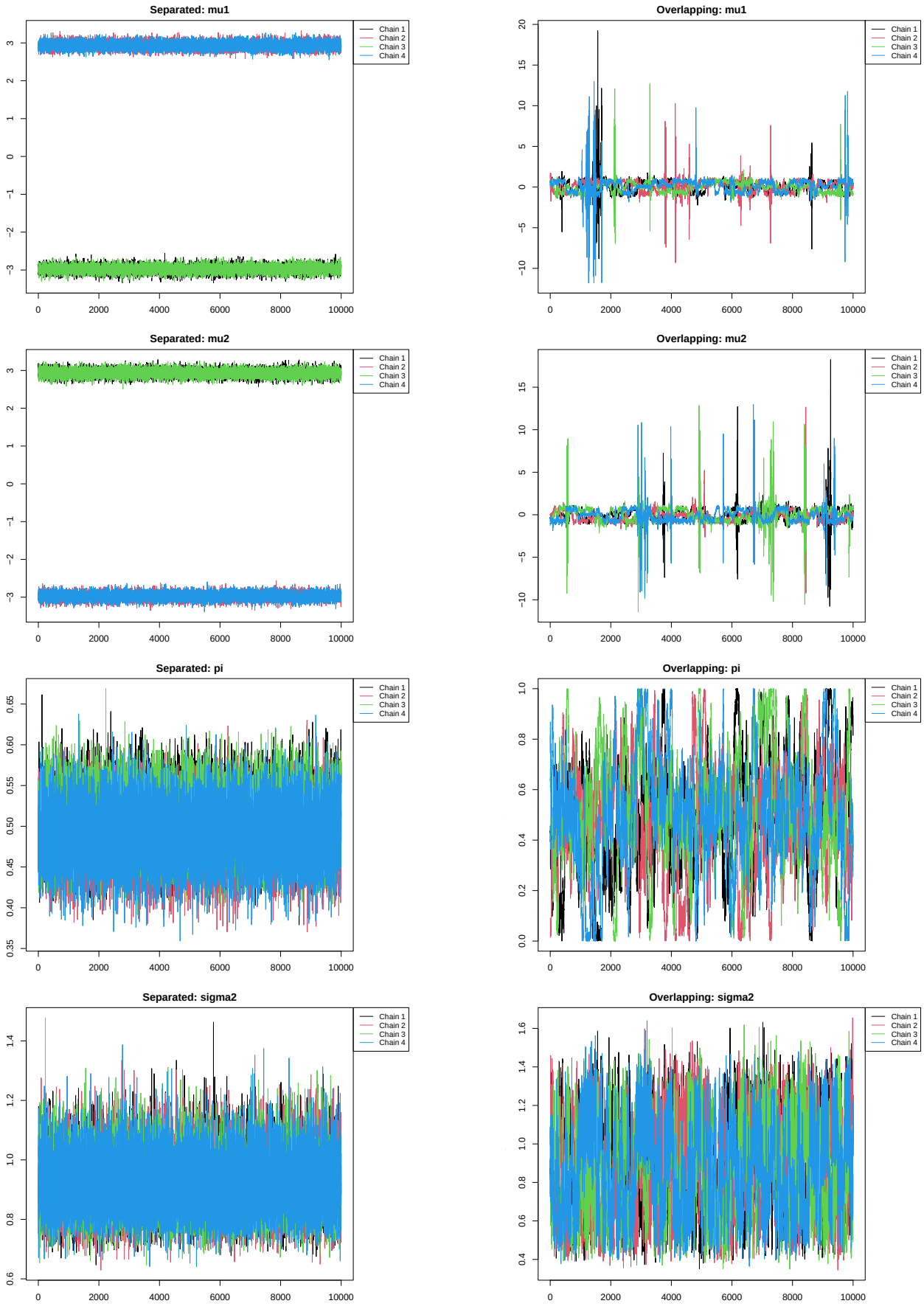
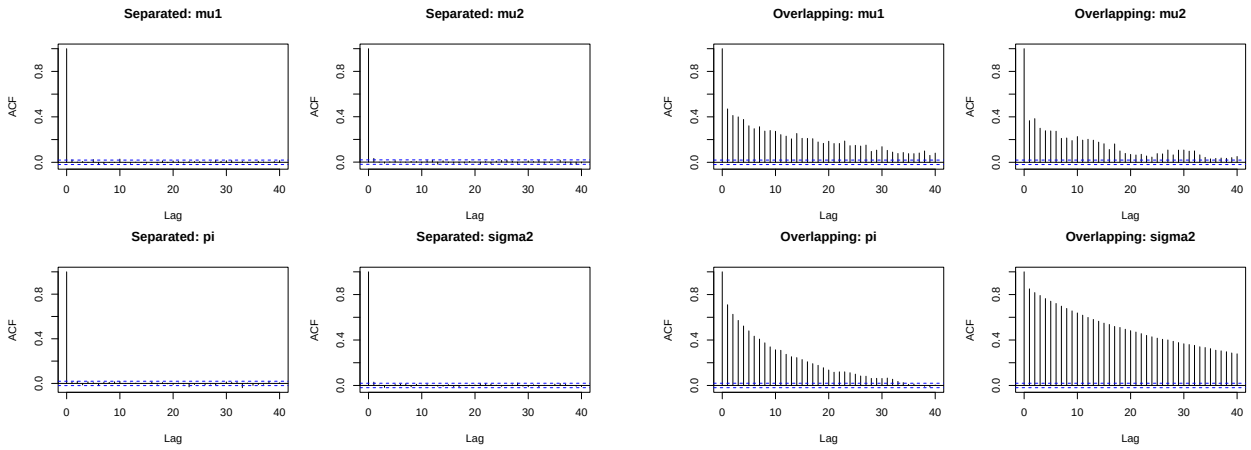


Figure 11: MCMC trace plots for the clearly separated (left column) and substantially overlapping (right column) Gaussian mixture models. The rows show the four chains for μ_1, μ_2, π, and σ^2, respectively.

Above, figure 11 shows the MCMC trace plots of four chains for both the clearly separated and substantially overlapping mixtures. In the clearly separated mixture, the parameter values remain relatively stable. However, the chains for μ_1 and μ_2 stay around two different values (the value is exchanged between the chains), due to label switching. In the overlapping mixture, the component means show large fluctuations, and the mixing probability π varies considerably between 0 and 1 over all the four chains. This suggest that the Gibbs sampler has greater difficulty estimating the parameters, when the components overlap substantially.

However, trace plots alone are insufficient to establish convergence. Therefore, we examined additional diagnostics[1], including $\hat{R}$, autocorrelation, effective sample size (ESS), and Monte Carlo standard error (MCSE).

Autocorrelation



(a) Clearly separated mixture

(b) Substantially overlapping mixture

Figure 12: Autocorrelation functions of μ_1, μ_2, π, and σ^2 for the clearly separated and overlapping Gaussian mixtures.

As shown in Figure 12, the autocorrelation[2] for the clearly separated mixture decreases rapidly towards zero for all parameters. This indicates relatively weak dependence between successive MCMC samples. But the overlapping mixture shows higher and slowly decreasing autocorrelation, which means that the Gibbs sampler mixes less efficiently when the components overlap substantially.

Numerical Convergence Diagnostics

In addition to autocorrelation, $\hat{R}$ [3], ESS[4], and MCSE[5] were calculated.

	$\hat{R}$		**ESS**		**MCSE**	
Parameter	Sep.	Over.	Sep.	Over.	Sep.	Over.
μ_1	1.000017	1.002011	9012.69	669.74	0.001008	0.022825
μ_2	1.000017	1.001946	9180.85	774.40	0.001016	0.021238
π	0.999974	1.000637	9843.30	518.86	0.000352	0.009991
σ^2	1.000043	1.002059	9469.09	113.54	0.000952	0.024387

Sep. = Clearly separated mixture; Over. = Substantially overlapping mixture.

Table 3: MCMC diagnostic results for the two Gaussian mixture models.

As shown in Table 3, the $\hat{R}$ values are very close to 1 for both mixtures, but Substantially overlapping mixture has a notably higher value compared to the Clearly separated mixture. The ESS values also show a clear difference. In the clearly separated mixture, all parameters have ESS values above 9,000, but the overlapping mixture has substantially lower ESS values. Similarly, the MCSE values are considerably higher in the overlapping mixture, indicating lower numerical precision in the estimated posterior means.

[1]Even though the ordering came on the last part of the problem, we have ordered the parameters for this part as well, because it will enable us to compare the same mixture component across different MCMC chains

[2]Measures the dependence between MCMC samples. So the lower autocorrelation means that the samples are less dependent on previous iterations.

[3]Compares the variation within and between MCMC chains. Values close to 1 are generally desirable.

[4]Estimates the amount of independent-equivalent information contained in the MCMC draws. A higher ESS indicates more efficient sampling.

[5]Measures the numerical uncertainty in the estimated posterior mean due to using a finite number of MCMC samples. Smaller MCSE values indicate more precise Monte Carlo estimates.

Overall, the autocorrelation, $\hat{R}$, ESS, and MCSE results suggest that the Gibbs sampler performs more efficiently for the clearly separated mixture.

2.5 Label Switching

Label switching occurs, because exchanging the labels of the mixture components does not change the overall probability distribution. For a two-component Gaussian mixture model, the density is

$$f(y \mid \theta) = \pi\varphi(y; \mu_1, \sigma^2) + (1 - \pi)\varphi(y; \mu_2, \sigma^2).$$

If the component labels are exchanged, the parameter values become

$$(\mu_1, \mu_2, \pi) \longrightarrow (\mu_2, \mu_1, 1 - \pi).$$

The corresponding density is

$$f(y \mid \theta) = (1 - \pi)\varphi(y; \mu_2, \sigma^2) + \pi\varphi(y; \mu_1, \sigma^2),$$

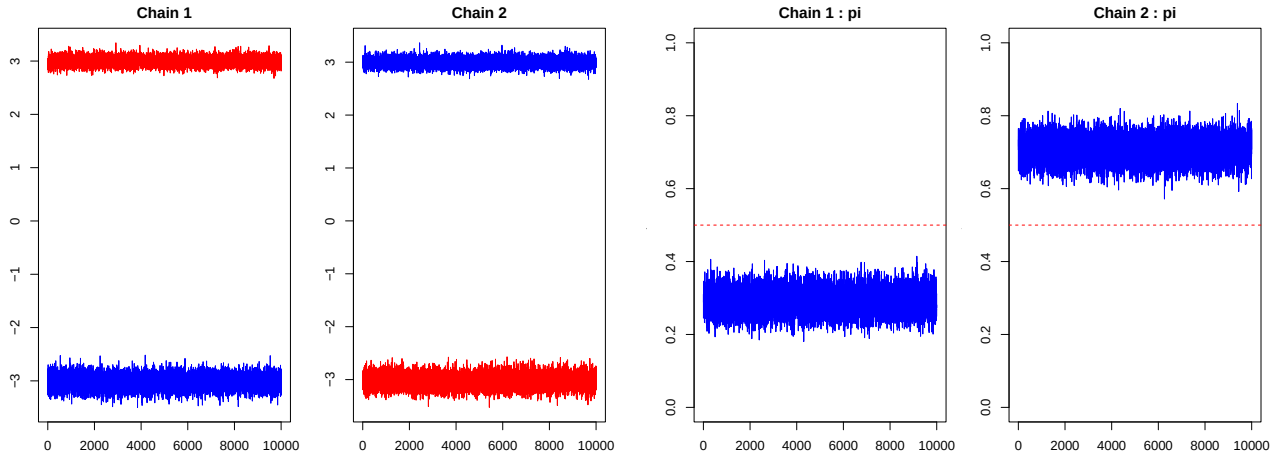
which is identical to the original mixture density.

2.5.1 Simulation of Label Switching and effect on Posterior Summaries

To investigate label switching, an additional dataset was generated with the below parameters[6]:

$$\mu_1 = -3, \qquad \mu_2 = 3, \qquad \sigma^2 = 1, \qquad \pi = 0.3.$$

The mixing probability was changed from 0.5 to 0.3 to make the two labelling arrangements easier to distinguish. Therefore, the first component has mean -3 and mixing probability 0.3, while the second component has mean 3 and mixing probability 0.7. The two chains were ran: chain 1 $(-0.5, 0.5)$ and chain 2 $(0.5, -0.5)$. Trace plots of μ_1, μ_2, and π are as follows.



(a) Component means (b) Mixing probability

Figure 13: MCMC trace plots illustrating label switching in the two-component Gaussian mixture model. The left panel shows the component means for two chains, while the right panel shows the corresponding mixing probabilities.

As shown in figure 13, in Chain 1, the first component mean is approximately -3 and the second is approximately $+3$, with a mixing probability around 0.3. However, in Chain 2, the component labels are reversed, and the mixing probability is approximately 0.7. Although the two chains use different parameter labels, they represent the same mixture distribution. This shows that different Gibbs sampler chains may assign different labels to the same components. This means that, when we calculate posterior means using samples from different labelling arrangements, we may average values from the lower mean and higher mean components together. As a result, the posterior means may not accurately represent the individual components.

[6]Due to the dataset that we have generated for the rest of the problem $\pi = 1 - \pi = 0.5$, which means we are unable to investigate the label switching in terms of mixture density

Example: Take the two components true parameters as:

$$\mu_1 = -3, \qquad \mu_2 = 3, \qquad \pi = 0.3$$

This means, Component 1 has mean -3 and weight 0.3. and Component 2 has mean 3 and weight 0.7. But the Gibbs sampler also can produce:

$$\mu_1 = 3, \qquad \mu_2 = -3, \qquad \pi = 0.7$$

These two sets of parameters describe exactly the same mixture distribution. Only the labels have changed.
Assume that the Gibbs sampler produces these posterior samples

Iteration	$\boldsymbol{\mu_1}$	$\boldsymbol{\mu_2}$	$\boldsymbol{\pi}$
1	-3	3	0.3
2	-3	3	0.3
3	3	-3	0.7
4	3	-3	0.7

Now if we calculate the posterior mean of each parameter.
For μ_1:

$$E(\mu_1 \mid y) \approx \frac{-3 - 3 + 3 + 3}{4} \approx 0$$

For μ_2:

$$E(\mu_2 \mid y) \approx \frac{3 + 3 - 3 - 3}{4} \approx 0$$

For π:

$$E(\pi \mid y) \approx \frac{0.3 + 0.3 + 0.7 + 0.7}{4} \approx 0.5$$

Our posterior summaries suggest:

$$\mu_1 \approx 0, \qquad \mu_2 \approx 0, \qquad \pi \approx 0.5$$

But the actual mixture contains:

$$\mu_1 = -3, \qquad \mu_2 = 3, \qquad \pi = 0.3$$

In our actual problem the histograms of the original component means μ_1 and μ_2, together with the re-ordered lower and higher component means, are as presented in Figure 14.

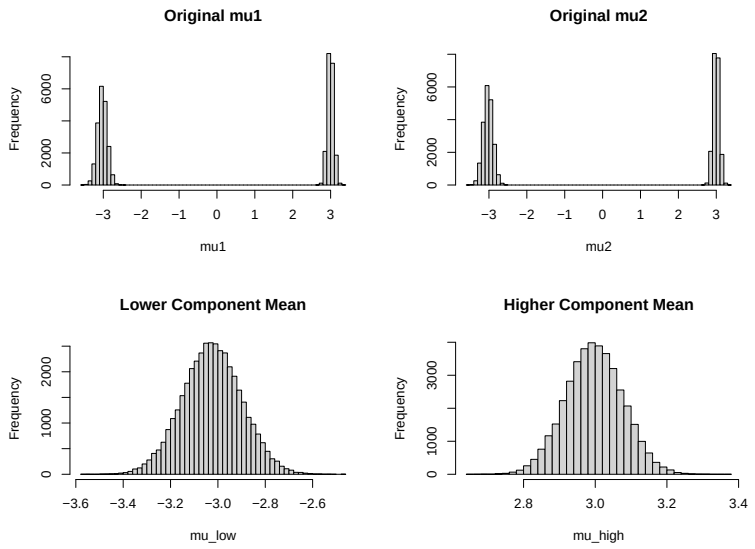


Figure 14: Posterior histograms of the original component means (top row) and the re-ordered component means (bottom row).

As shown in Figure 14, the original posterior distributions of μ_1 and μ_2 have two peaks, approximately at -3 and 3, due to label switching. After ordering the component means using $\mu_{\text{low}} = \min(\mu_1, \mu_2)$ and $\mu_{\text{high}} = \max(\mu_1, \mu_2)$, the resulting distributions have a single peak around -3 and 3, respectively. This shows that ordering the component means allows us to obtain more meaningful component-specific posterior summaries, without changing the underlying mixture model.

Table 4: Posterior means before and after label ordering

	μ_1 / μ_{low}	μ_2 / μ_{high}	π / π_{low}	σ^2
Before ordering	-0.0497	-0.0483	0.5000	0.9770
After ordering	-3.1136	3.0157	0.3018	0.9770

As for the numerical posterior summaries, Table 4 compares the posterior means before and after label ordering. The posterior means of μ_1 and μ_2 are close to zero before label ordering. This happens because the samples from the two labelling arrangements are averaged together. After ordering, the posterior means become approximately -3.11 and 3.02, which are close to the true component means of -3 and 3. The adjusted mixing probability is approximately 0.30, which is also close to its true value. The estimate of the common variance, however, remains unchanged because exchanging component labels does not affect σ^2.

2.5.2 Quantities not affected by the label exchange

As explained at the beginning of this section, switching the component labels does not change the overall mixture density. Similarly, the overall mixture variance remains unchanged under label switching[7]. The overall mixture variance is given by

$$\text{Var}(Y) = \sigma^2 + \pi(1-\pi)(\mu_1 - \mu_2)^2.$$

In addition, the overall mixture mean is also unaffected by label switching. The mixture mean is given by

$$E(Y) = \pi\mu_1 + (1-\pi)\mu_2.$$

After exchanging the component labels, the mean becomes

$$E(Y) = (1-\pi)\mu_2 + \pi\mu_1,$$

which is same as the original expression.

Therefore, although label switching affects the interpretation of individual component parameters, the overall mixture density, mean, and variance remain unchanged.

2.6 Application to the `faithful` data and posterior interpretation

2.6.1 Data, sampler runs and label switching

We applied the sampler to the `eruptions` variable from the `faithful` dataset ($n = 272$). The data were standardised with a mean of 3.488 and a standard deviation of 1.141 minutes. The results presented below have been converted back to their original units (minutes).

The prior distributions used are consistent with those specified in section 2.2.2: $\mu_1, \mu_2 \sim N(0, 5^2)$, $\pi \sim \text{Beta}(1, 1)$ and $\sigma^2 \sim \text{InvGamma}(2, 1)$. These priors are weakly informative, since they cover a much wider range than the standardised data. We ran 4 Markov chains, each with 12 000 iterations and using different initial values and the first 2000 iterations of each chain were dropped.

No chain switched labels within the run, because the two components are well separated. However, chains 2 and 4 stayed in the swapped labelling ($\mu_1 > \mu_2$ and $\pi \approx 0.64$), because they started from swapped values, as shown in Table 5. Therefore, simply merging the chains would mix these two components. In every iteration with $\mu_1 > \mu_2$, the means were swapped, the weights of lower components were set to $1 - \pi$, and the assignments for Z_i were switched to ensure that the lower component always corresponds to a smaller mean.

	Mean of μ_1	Mean of μ_2	Mean of π	Label changes
Chain 1	-1.262	0.709	0.361	0
Chain 2	0.709	-1.262	0.639	0
Chain 3	-1.261	0.710	0.361	0
Chain 4	0.709	-1.261	0.639	0

Table 5: Posterior means by chain before relabelling, on the standardised scale. Label changes is the number of iterations at which the order of μ_1 and μ_2 changes within a chain.

[7]Exchanging the component labels does not change either $\pi(1-\pi)$ or $(\mu_1 - \mu_2)^2$.

2.6.2 Posterior summaries and MCMC performance

Table 6 provides posterior summary results and diagnostics. The $\hat{R}$ value for all parameters is 1.00. The traces of the 4 chains overlap. Among the 40 000 saved sampling samples, the effective sample size exceeds 36 000, indicating that these sampling samples are almost uncorrelated to each other.

	Mean	Standard deviation	2.5% quantile	97.5% quantile	$\hat{R}$	ESS
μ_{low}	2.048	0.039	1.972	2.124	1.00	36 086
μ_{high}	4.297	0.029	4.241	4.353	1.00	37 964
π_{low}	0.361	0.029	0.305	0.419	1.00	38 646
σ	0.377	0.016	0.346	0.411	1.00	36 967

Table 6: Posterior summaries of the relabelled parameters (four chains, 40,000 draws). *Low* and *high* denote the components with the smaller and larger mean, π_{low} is the weight of the low component, and σ is the common standard deviation. Means and σ are in minutes.

The lower component has mean 2.048 minutes and weight 0.361, and the upper component has mean 4.297 minutes and weight 0.639. Both components share a standard deviation of 0.377 minutes.

2.6.3 Fitted mixture and uncertainty of the allocations

Figure 15 shows the fitted mixture density $\pi f_{\text{low}} + (1-\pi) f_{\text{high}}$ together with the histogram and a kernel density estimate. It has two peaks at about 2.0 and 4.3 minutes, which match the two modes of the histogram, and it is close to zero in the gap between them.

Compared with the kernel density estimate (red), the posterior mean mixture (blue) follows the histogram more closely around the higher peak, and it is closer to zero in the gap near 3.2 minutes, where the kernel density estimate is smoothed upwards. The fit is not perfect. The highest bar of the short eruptions (near 1.8 minutes) is above the fitted density, and the fitted density lies below the histogram between about 3.3 and 3.6 minutes.

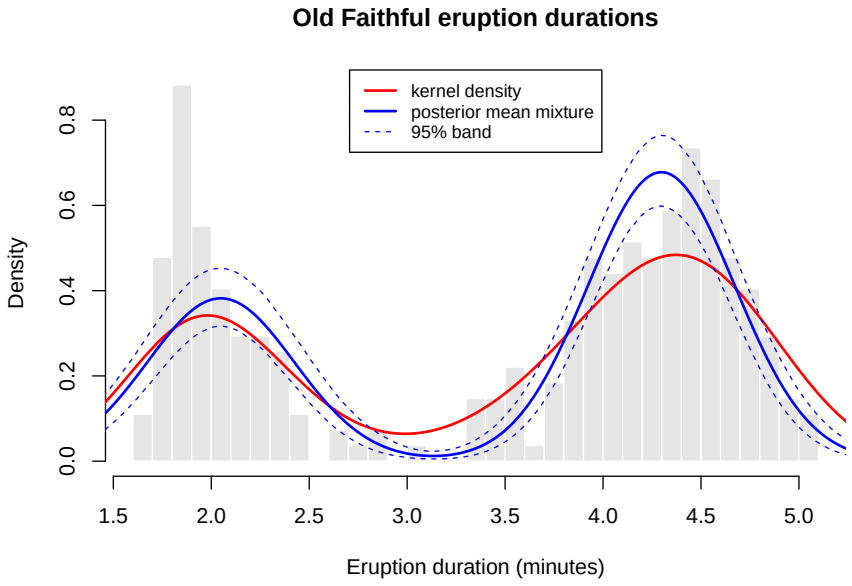


Figure 15: Histogram, kernel density estimate (red) and posterior mean mixture density with a pointwise 95% band (blue) for the eruption durations.

To evaluate the uncertainty of allocation, for each observation, the share of draws that allocate it to the upper component gives its posterior probability of belonging to that component (Figure 16). In total, 174 of the 272 observations have a probability above 0.5, which agrees with the estimated weight of 0.64. Only 2 observations have a probability between 0.05 and 0.95, at durations of 3.07 and 3.32 minutes, which lie in the gap between the two groups. Therefore, the uncertainty of allocation is limited to the boundary regions between components.

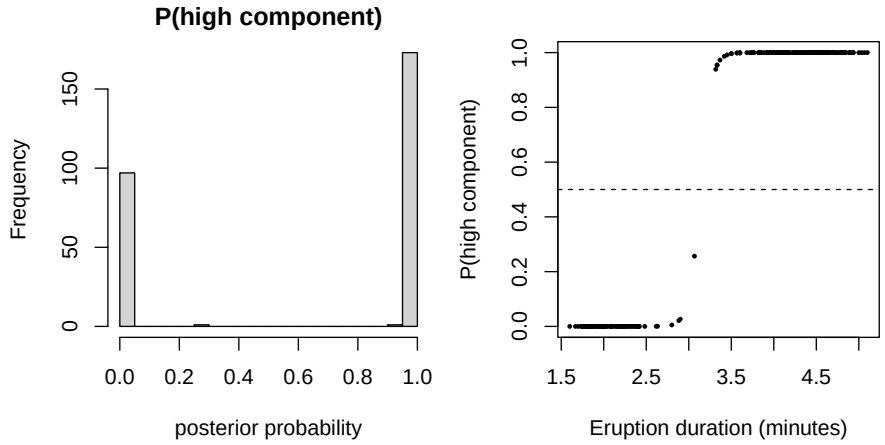


Figure 16: Posterior probability that an observation belongs to the high component: histogram over all observations (left) and probability against eruption duration (right). The dashed line marks 0.5.

2.6.4 Model adequacy and interpretation of the components

The two-component Gaussian mixture is an adequate description of the eruption durations, although it fits the short eruptions worse.

A posterior predictive check was used. For 500 posterior draws, datasets with a sample size of 272 were simulated using the fitted model; the sample skewness and excess kurtosis of these simulated datasets were then compared with the observed values. The observed skewness (-0.414) and excess kurtosis (-1.512) are not extreme, with corresponding posterior predictive p-values of 0.318 and 0.866. The kurtosis of the actual data falls toward the lower end of the distribution of the simulated values, implying that the simulated data has relatively heavier tails.

The lower component describes short eruptions of about 2 minutes, and the upper component describes longer eruptions of about 4.3 minutes. However, while these components describe the shape of the distribution, they do not necessarily show that these eruptions come from two physically distinct populations.

2.7 Conclusions

In this investigation, Gibbs sampling was used for Bayesian mixture models. The simulation results showed that the Gibbs sampler performed better when the two components were clearly separated. When the components overlapped substantially, estimating the parameters became more difficult. This conclusion was also supported by the autocorrelation, ESS, and MCSE results.

Also, when it comes to the label switching issue, the results showed that exchanging component labels does not change the overall mixture distribution, but it can lead to misleading posterior estimates of individual parameters. By ordering the component means, we were able to obtain more meaningful posterior summaries. Therefore, the ordering became a crucial part of this problem.

In the application to the `faithful` data, the sampler gave a short-eruption component of about 2 minutes (weight 0.36) and a long-eruption component of about 4.3 minutes (weight 0.64). The chains converged well after relabelling, and almost all observations had a clear allocation. Posterior predictive checks revealed no significant conflict with the model. The mixture model effectively captured the distribution of eruption durations. However, this does not necessarily imply that these eruptions come from two physically distinct populations.

References

[1] Nordhausen, K. and Virta, J. (2018). Ladle estimator for time series signal dimension. *2018 IEEE Statistical Signal Processing Workshop (SSP)*, 428–432.

[2] Politis, D. N. and Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313.